

# claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-ac12a4686197057fc.jsonl`  
- SHA-256: `ddc7f0714af0ae233344874e94e7781635dadf895a044147bbfbdabda1909444`  
- Source modified: `2026-07-08T06:17:49+00:00`  
- Imported at: `2026-07-08T15:59:28+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

## Transcript

### 1. user (2026-07-08T06:14:14.453Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/\_\_main\_\_.py, undefined/backend/compute/cloud\_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase\_placement config and compute\_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download\_url, video\_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration.\_\_stitch\_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud\_run\_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses \_compile/, detect uses \_dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is\_real=false if you cannot concretely confirm the failure. Skeptic #2.

CLAIM (medium/test-coverage) backend/worker/\_\_\_main\_\_.py:865  
summary: The dispatcher->worker contract (compile argv flags and the staged spec JSON schema) is never round-tripped end-to-end; each side is tested against its own hand-built fixture, so a flag/key drift between them passes CI but breaks prod.  
failure scenario: CloudRunCompute.run\_compile (cloud\_run.py:323) emits argv ["compile", "--out-uri", ..., "--spec-uri", ..., "--done-uri", ..., "--set", ...], and \_dispatch\_compile\_remote stages a spec with keys {video\_id, video\_uri, output\_name, locale, time\_pairs, stitching} (orchestration.py:1330-1348); cmd\_compile parses that argv via build\_parser()'s compile subparser (worker/\_\_\_main\_\_.py:865) and reads those exact spec keys (worker/\_\_\_main\_\_.py:490-496). No test connects the two halves: test\_run\_compile\_dispatches\_and\_returns\_marker asserts run\_compile's emitted argv via a fake client that never calls build\_parser(), and test\_parser\_compile\_accepts\_spec\_and\_done\_uris parses a HAND-WRITTEN argv; likewise the spec is written by test fixtures independent of the CP producer. If a flag is renamed/removed or made required on only one side (e.g. --spec-uri), or a spec key drifts (video\_uri vs source\_uri, time\_pairs), both tests still pass, but the real worker exits 2 on an unrecognized/missing-required arg (no marker -> opaque failed job via crash-fast) or KeyErrors reading the spec. The duplicated string literals across producer and consumer make this drift easy to introduce and currently unguarded.

## 2. assistant (2026-07-08T06:14:29.039Z)

I'll start by finding the actual code files since the paths came through as "undefined". Let me explore the repository structure and pull latest.

## 3. user (2026-07-08T06:14:31.613Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)  
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells (§9) (#522)  
699d85b feat(validation): cache validation verdict by video\_id to skip re-validation on reupload (#518)  
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)  
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)  
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)  
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)  
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)  
---REMOTE---  
chore/revendor-hosted-compute-picker  
+ claude/compile-reliability  
+ claude/wasb-config-validation  
\* codex/issue-523-sd-reel-plan  
+ feat/521-compile-on-l4  
+ feat/524-l4-primary  
feat/submit-md5-cache-reuse  
fix/cloud-run-exec-status-fastfail  
fix/hosted-local-detection-guardrail  
master  
+ perf/521-cheap-watermark  
perf/validation-shared-decode-pass  
remotes/origin/HEAD -> origin/master  
remotes/origin/chore/revendor-hosted-compute-picker  
remotes/origin/docs/runlog-cloud-run-exec-status-fastfail  
remotes/origin/docs/validation-latency-live-cuj-2026-07-08  
remotes/origin/feat/521-compile-on-l4  
remotes/origin/feat/validation-result-cache  
remotes/origin/fix/cloud-run-exec-status-fastfail  
remotes/origin/fix/hosted-local-detection-guardrail

## 4. user (2026-07-08T06:14:33.031Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521)  
428ee44 address follow-up review on #169: correct factual error in item-3 investigation note (fusion\_compare.py, fusion\_golden.py, ablation.py + tests \*do\* exist per ls/grep on the tree; person features in fusion\_hybrid + detectors/fusion\_features.py; remaining M0 in-container task is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan now accurate. M0 item 3 placement remains correct and even stronger.  
318f6c2 test: positive tests for Item 2 (config) - shallow section replacement, unknown keys warning, get shim, roundtrip + living doc update for T2 post #157  
8836dfe chore: scripts/ for run-drivers + CODE\_MAP \$0 file-placement guidelines (#138)

## 5. assistant (2026-07-08T06:14:39.830Z)

The PR #521 merged as commit `dac6736`. Let me read the relevant files at that commit and the diff. Let me look at the actual code and tests.

## 6. user (2026-07-08T06:14:41.386Z)

commit dac6736cb23a681135b707d768014820b239a981  
Author: Avi <avi.dullu@gmail.com>  
Date: Wed Jul 8 11:29:33 2026 +0530

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, \$9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on \_LOCAL\_COMPUTE\_LANE, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: deployment.phase\_placement {validation|segment|compile} ? control\_plane | cloud\_run\_l4, resolved by backend/config/placement.py. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a kind='compile' Cloud Run dispatch mirroring \_maybe\_dispatch\_remote. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches `worker compile` on the L4, bucket-polls a done-marker, and returns the same {status, filepath, download\_url} shape the SPA expects. The worker stitches via the SHARED \_stitch\_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud\_stitch\_encoder (default h264\_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase\_placement.validation moves the validator suite off the control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE\_DECOUPLED\_SERVING/PHASE\_PLACEMENT.md (+ README pointer).  
Tests: new test\_phase\_placement.py, test\_compile\_dispatch.py; additions to test\_compute\_backend.py (run\_compile), test\_worker.py (cmd\_compile), test\_compiler.py (encoder selection), test\_api\_cloud\_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

|                              |            |
|------------------------------|------------|
| backend/compute/__init__.py  | 2 +        |
| backend/compute/base.py      | 34 +++     |
| backend/compute/cloud_run.py | 100 ++++++ |
| backend/config/__init__.py   | 16 ++      |
| backend/config/models.py     | 52 ++++    |

```

backend/config/placement.py | 59 ++++
backend/config/presets.py | 10 +-
backend/orchestration.py | 322 ++++++-----
backend/pipeline/stitcher/compiler.py | 94 +++++-
backend/worker/__main__.py | 139 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md | 169 ++++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 1 +
tests/test_api_cloud_dispatch.py | 70 +++++
tests/test_compile_dispatch.py | 204 ++++++
tests/test_compiler.py | 64 +++++
tests/test_compute_backend.py | 111 ++++++
tests/test_config_deployment.py | 5 +
tests/test_phase_placement.py | 138 ++++++
tests/test_worker.py | 175 ++++++

```

## 7. user (2026-07-08T06:14:41.807Z)

codex/issue-523-sd-reel-plan

## 8. assistant (2026-07-08T06:14:48.788Z)

Current branch differs from the merge. Let me read the exact post-#521 code from the merge commit. I'll focus on the producer (cloud\_run.py, orchestration.py), consumer (worker/\_\_main\_\_.py), and the tests.

## 9. user (2026-07-08T06:14:53.288Z)

```

516 /tmp/cloud_run.py
977 /tmp/worker_main.py
1460 /tmp/orchestration.py
204 /tmp/test_compile_dispatch.py
1120 /tmp/test_worker.py
1213 /tmp/test_compute_backend.py
5490 total

```

## 10. assistant (2026-07-08T06:14:56.113Z)

Let me read the producer side (cloud\_run.py run\_compile) and the worker's compile parser and cmd\_compile.

## 11. user (2026-07-08T06:14:57.582Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer.

## 12. user (2026-07-08T06:14:57.647Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer.

## 13. assistant (2026-07-08T06:15:00.601Z)

The `/tmp` path doesn't map for the Read tool on Windows. Let me write to the scratchpad directory.

## 14. user (2026-07-08T06:15:06.196Z)

```

total 1416
drwxr-xr-x 1 avidu 197609      0 Jul  8 11:45 .
drwxr-xr-x 1 avidu 197609      0 Jul  8 10:35 ..
-rw-r--r-- 1 avidu 197609 25174 Jul  8 11:45 cloud_run.py
-rw-r--r-- 1 avidu 197609 25174 Jul  8 11:36 cloud_run_521.py
-rw-r--r-- 1 avidu 197609 25174 Jul  8 11:35 cloudrun_525.py

```

```

-rw-r--r-- 1 avidu 197609 28017 Jul 8 11:36 compiler_521.py
-rw-r--r-- 1 avidu 197609 28017 Jul 8 11:35 compiler_525.py
drwxr-xr-x 1 avidu 197609 0 Jul 8 11:30 compiletest
-rw-r--r-- 1 avidu 197609 4085 Jul 8 11:35 compute_init_521.py
-rw-r--r-- 1 avidu 197609 4085 Jul 8 11:37 compute_init_525.py
-rw-r--r-- 1 avidu 197609 75134 Jul 8 11:42 main_525.py
-rw-r--r-- 1 avidu 197609 36917 Jul 8 11:35 models_525.py
-rw-r--r-- 1 avidu 197609 71951 Jul 8 11:36 orch_521.py
-rw-r--r-- 1 avidu 197609 71951 Jul 8 11:35 orch_525.py
-rw-r--r-- 1 avidu 197609 71951 Jul 8 11:36 orch_post.py
-rw-r--r-- 1 avidu 197609 71951 Jul 8 11:45 orchestration.py
-rw-r--r-- 1 avidu 197609 71951 Jul 8 11:35 orchestration_521.py
-rw-r--r-- 1 avidu 197609 71951 Jul 8 11:42 orchestration_525.py
-rw-r--r-- 1 avidu 197609 71951 Jul 8 11:44 orchestration_merged.py
-rw-r--r-- 1 avidu 197609 102148 Jul 8 11:34 pr521.diff
drwxr-xr-x 1 avidu 197609 0 Jul 8 11:37 pr525
-rw-r--r-- 1 avidu 197609 104789 Jul 8 11:34 pr525.diff
-rw-r--r-- 1 avidu 197609 54822 Jul 8 11:34 pr525_backend.diff
drwxr-xr-x 1 avidu 197609 0 Jul 8 11:31 review
-rw-r--r-- 1 avidu 197609 9276 Jul 8 11:43 storage_init_525.py
-rw-r--r-- 1 avidu 197609 8939 Jul 8 11:41 t_compile_dispatch.py
-rw-r--r-- 1 avidu 197609 39459 Jul 8 11:41 t_worker.py
-rw-r--r-- 1 avidu 197609 8939 Jul 8 11:45 test_compile_dispatch.py
-rw-r--r-- 1 avidu 197609 50335 Jul 8 11:45 test_compute_backend.py
-rw-r--r-- 1 avidu 197609 39459 Jul 8 11:45 test_worker.py
-rw-r--r-- 1 avidu 197609 3580 Jul 8 11:36 watermark_i18n.py
-rw-r--r-- 1 avidu 197609 40742 Jul 8 11:36 worker_521.py
-rw-r--r-- 1 avidu 197609 40742 Jul 8 11:35 worker_525.py
-rw-r--r-- 1 avidu 197609 40742 Jul 8 11:45 worker_main.py
-rw-r--r-- 1 avidu 197609 40742 Jul 8 11:42 worker_main_525.py
-rw-r--r-- 1 avidu 197609 40742 Jul 8 11:36 worker_post.py

```

#### 15. assistant (2026-07-08T06:15:10.579Z)

Files are written. Let me read the producer's `run\_compile` in cloud\_run.py and the worker's compile parser + cmd\_compile.

#### 16. user (2026-07-08T06:15:13.156Z)

```

290         "video_id) ? treating as failure",
291         self._job_name,
292     )
293     rc = rc or 1
294     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
295     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
296     LOG.info(
297         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
298     )
299     return ComputeResult(
300         returncode=rc,
301         outputs_uri=outputs_uri,
302         video_id=video_id,
303         report=report,
304         execution=str(execution or ""),
305     )
306
307     def run_compile(
308         self,
309         spec: "CompileJobSpec",
310         *,
311         on_wait: "Callable[[float], None] | None" = None,
312     ) -> ComputeResult:
313         """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
314         bucket-poll the done-marker, and return the worker's rc + marker. Reuses the

```

```

EXACT
315         dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast
+ timeout
316         rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
317         ~65-min opaque hang.""
318         out_root = spec.out_uri.rstrip("/")
319         token = self._token or uuid.uuid4().hex
320         # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
321         # bucket can never collide on a marker.
322         done_uri = f"{out_root}/_compile/{token}.done"
323         argv: List[str] = [
324             "compile",
325             "--out-uri",
326             spec.out_uri,
327             "--spec-uri",
328             spec.spec_uri,
329             "--done-uri",
330             done_uri,
331         ]
332         # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
333         # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
334         for kv in spec.config_overrides:
335             argv += ["--set", kv]
336
337         execution = self._client.run_job(
338             self._job_name, argv, region=self._region, project=self._project
339         )
340         LOG.info(
341             "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
342             self._job_name,
343             execution,
344             done_uri,
345         )
346         marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
347         video_id = str(marker.get("video_id", "")) or spec.video_id
348         rc = int(marker.get("returncode", 1))
349         LOG.info(
350             "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
351             self._job_name,
352             video_id,
353             rc,
354         )
355         # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
356         # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
357         # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
358         return ComputeResult(
359             returncode=rc,
360             outputs_uri=str(marker.get("reel_uri", "")) or "",
361             video_id=video_id,
362             report=marker,
363             execution=str(execution or ""),
364         )
365
366     def _await_marker(
367         self,
368         done_uri: str,
369         execution: object,
370         on_wait: "Callable[[float], None] | None",

```

```

371         *,
372         label: str,
373     ) -> dict:
374         """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
375         ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting tick
reaches the
376         caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
377         ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
378         maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
379         try:
380             return poll_until(
381                 lambda: self._poll_check(done_uri, str(execution or "")),
382                 self._policy,
383                 label=label,
384                 logger=LOG,
385                 on_tick=on_wait,
386             )
387         except PolicyTimeout as timeout_exc:
388             return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
389
390     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
391         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
392
393         The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
394         wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
395         FAILURE* marker resolves through the normal path with its real returncode. Only
when the
396         marker is absent do we consult the execution status: a worker that TERMINATED
without ever
397         writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
398         otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
399         opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable

```

## 17. user (2026-07-08T06:15:14.226Z)

```

440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
444         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
445         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
446         try:
447             marker = {
448                 "video_id": video_id,
449                 "returncode": returncode,
450                 "status": result.get("status", "failed"),
451                 "download_url": result.get("download_url", ""),
452                 "reel_uri": result.get("reel_uri", ""),
453                 "message": result.get("message", ""),
454             }
455             local = os.path.join(scratch, "_compile_done.json")
456             with open(local, "w", encoding="utf-8") as f:
457                 json.dump(marker, f)
458             storage.put(local, done_uri, config=storage_cfg)

```

```

459         logger.info("[worker] wrote compile marker -> %s", done_uri)
460     except Exception as e: # never fail a good stitch because the marker push hiccuped
461         logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
462 e)
463
464     def cmd_compile(args: argparse.Namespace) -> int:
465         """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
466 stitch is the
467         2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
468 4K reel). Read
469         the control-plane-staged compile-spec (resolved time-pairs + stitching config +
470 source URI), stitch
471         the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
472 there is no
473         forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
474 the dispatcher's
475         bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
476 requires."""
477         from backend import orchestration
478         from backend.config import StitchingConfig
479
480         config = load_config(args.config) if args.config else load_config()
481         _apply_overrides(config, args.set)
482         storage_cfg = config.storage.model_dump()
483
484         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
485         os.makedirs(scratch, exist_ok=True)
486         # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
487 the bucket.
488         config.output.output_dir = scratch
489         # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
490 value as a
491         # fallback so reel_object_uri (which reads storage.output_root) always resolves the
492 reel's bucket
493         # destination even if the override were dropped.
494         if not (config.storage.output_root or "").strip():
495             config.storage.output_root = args.out_uri
496
497         spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
498         video_id = str(spec.get("video_id") or "")
499         out_name = str(spec.get("output_name") or "")
500         source_ref = str(spec.get("video_uri") or "")
501         locale = spec.get("locale")
502         time_pairs = [
503             (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
504         ]
505         stitching = StitchingConfig(**(spec.get("stitching") or {}))
506
507         print(
508             f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
509             f"encoder={stitching.encoder} out={out_name}"
510         )
511
512         result = orchestration._stitch_reel(
513             config,
514             stitching,
515             video_id,
516             source_ref,
517             time_pairs,
518             out_name,
519             locale,
520             lambda stage: print(f"[worker] compile: {stage}"),
521         )
522         status = str(result.get("status") or "failed")

```



```

514         rc = 0 if status == "success" else 3
515         if status != "success":
516             print(f"[worker] compile FAILED: {result.get('message')}")
517
518         # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
519         if getattr(args, "done_uri", None):
520             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
521         return rc
522
523
524         #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
525         #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
526         _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
527
528         #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
529         #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
530         #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
531         _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
532
533
534         def _label_clip_name(fname: str) -> str:
535             """Derive the clip name (the `videos/<name>.<ext>` join key) from a label
filename.
536
537             Strips both the `.rallies.csv` / `.csv` suffix and an optional leading ISO-date
prefix, so a
538             canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
539
540             * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-
stamped)
541             * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same
stem)
542             * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of
the name, kept)
543             * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in
<name> survive)
544
545             Without the date strip, `--trajectory-source detector` can't match a dated label
to its bare
546             video stem, so every clip is skipped and train aborts with `<2 videos`
(DATA_IN_GCS §7b #1).
547             """
548             name = fname.replace(".rallies.csv", "").replace(".csv", "")
549             return _LABEL_DATE_PREFIX.sub("", name)
550
551
552         def _probe_video_fps_width(path: str):
553             """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
554
555             The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
556             fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
557             cv2's `_probe_fps_width` silently defaults to (30, 1280) on a read miss ? fine for
the
558             inference report (it flags the fallback) but corrupting for REAL training data. So
here we
559             probe with ffprobe (same tool as `get_video_duration`) and return None so the
caller SKIPS

```

## 18. assistant (2026-07-08T06:15:24.396Z)

Now let me find the compile subparser in build\_parser and the spec-staging in orchestration.

## 19. user (2026-07-08T06:15:28.738Z)

```
422: def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
423:     """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
    ``time_pairs``, the
427:         spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
432:         raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
436: def _write_compile_marker(
444:     """#521: write the compile done-marker (rc + status + the reel's download_url/reel_uri)
    so the
455:         local = os.path.join(scratch, "_compile_done.json")
459:         logger.info("[worker] wrote compile marker -> %s", done_uri)
461:         logger.warning("[worker] could not write compile done-marker %s: %s", done_uri, e)
464: def cmd_compile(args: argparse.Namespace) -> int:
467:     the control-plane-staged compile-spec (resolved time-pairs + stitching config + source
    URI), stitch
478:     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
488:     spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
499:     f"[worker] compile: video_id={video_id} clips={len(time_pairs)}" "
511:     lambda stage: print(f"[worker] compile: {stage}"),
516:     print(f"[worker] compile FAILED: {result.get('message')}")
520:     _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
531: LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
809: def build_parser() -> argparse.ArgumentParser:
814:     p.add_argument(
822:         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
823:         pi.add_argument(
826:         pi.add_argument(
831:         pi.add_argument(
834:         pi.add_argument(
837:         pi.add_argument(
840:         pi.add_argument("--max-frames", type=int, default=None)
841:         pi.add_argument(
848:         pi.add_argument(
855:         pi.add_argument(
865:         pc = sub.add_parser(
866:             "compile",
867:             help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged compile-
            spec",
869:         pc.add_argument(
872:             help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
            stitching "
875:         pc.add_argument(
881:         pc.add_argument(
884:             help="storage URI for the compile completion MARKER (rc + download_url), so a remote
            "
887:         pc.add_argument(
890:         pc.add_argument(
893:         pc.add_argument(
900:         pc.set_defaults(func=cmd_compile)
902:         pt = sub.add_parser(
906:         pt.add_argument(
912:         pt.add_argument(
917:         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
918:         pt.add_argument(
926:         pt.add_argument(
931:         pt.add_argument(
934:         pt.add_argument("--config", default=None)
935:         pt.add_argument("--scratch", default=None)
936:         pt.add_argument(
942:         pt.add_argument(
```

```
948: pt.add_argument("--set", action="append", default=[], metavar="k=v")
971: args = build_parser().parse_args(argv)
```

## 20. user (2026-07-08T06:15:31.101Z)

```
393:         video_path = str(report.get("video_uri") or "") or _first_object_under(
588:             storage.put(
670:                 video_uri=req.video_path,
671:                 out_uri=out_root,
673:                 config_overrides=tuple(overrides),
1105:     and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known shot_count
are exempt
1122:     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1128:         if sc is not None and int(sc) < stitching.min_shot_count:
1135:             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1136:             stitching.min_shot_count,
1144:             f"stitching.min_shot_count={stitching.min_shot_count}.",
1186:         return "stitching on cloud GPU (NVENC) ? starting up"
1187:     return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1190: def _stitch_reel(
1192:     stitching,
1195:     time_pairs: List[Tuple[float, float]],
1201:     (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1203:     with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1214:     localized_watermark = localize_watermark(stitching.watermark, locale)
1217:     begin_padding=stitching.begin_padding,
1218:     end_padding=stitching.end_padding,
1220:     encoder=getattr(stitching, "encoder", "libx264"),
1223:     notify("stitching")
1235:     out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1238:     "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1242:     "message": f"Highlight stitching failed: {e}",
1262:     storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1279: def _dispatch_compile_remote(
1290:     small compile-spec (resolved time-pairs + the resolved stitching/watermark config + the
source URI)
1295:     from backend.compute import CompileJobSpec, get_compute_backend
1305:     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1315:     "stitching in-process on the control-plane instead.",
1318:     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1320:     cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
1324:     # the stitching config is the control-plane's RESOLVED one, with the encoder swapped to
the L4's
1326:     stitch_dump = stitching.model_dump()
1332:     "video_uri": video["filepath"],
1333:     "output_name": out_name,
1335:     "time_pairs": [
1338:     "stitching": stitch_dump,
1341:     spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1348:     storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1351:     "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
1365:     spec = CompileJobSpec(
1366:         out_uri=out_root,
1367:         spec_uri=spec_uri,
1369:         output_name=out_name,
1370:         config_overrides=tuple(overrides),
1431:     (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
1444:     return _dispatch_compile_remote(
1456:     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
```

```
1457:     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1459:         cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
```

## 21. assistant (2026-07-08T06:15:33.643Z)

Let me read the compile subparser definition and the spec-staging block in orchestration precisely.

## 22. user (2026-07-08T06:15:37.223Z)

```
865         pc = sub.add_parser(
866             "compile",
867             help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
868         )
869         pc.add_argument(
870             "--spec-uri",
871             required=True,
872             help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
873                 "config + source uri + video_id)",
874         )
875         pc.add_argument(
876             "--out-uri",
877             default="",
878             help="output ROOT (gs:/// bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
879                 "Also forwarded via --set storage.output_root; either resolves the reel path.",
880         )
881         pc.add_argument(
882             "--done-uri",
883             default=None,
884             help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
885                 "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
886         )
887         pc.add_argument(
888             "--config", default=None, help="config.json path (default: ./config.json)"
889         )
890         pc.add_argument(
891             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
892         )
893         pc.add_argument(
894             "--set",
895             action="append",
896             default=[],
897             metavar="k=v",
898             help="config override, e.g. --set storage.output_root=gs://bucket",
899         )
900         pc.set_defaults(func=cmd_compile)
901
902         pt = sub.add_parser(
903             "train",
904             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
```

## 23. user (2026-07-08T06:15:38.120Z)

```
1279     def _dispatch_compile_remote(
1280         cfg,
1281         db,
1282         video_id: str,
1283         segment_ids: List[int],
1284         output_filename: str,
```

```

1285     locale: Optional[str],
1286     notify,
1287 ) -> Dict[str, Any]:
1288     """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
1289     control-plane owns the DB, so it RESOLVES the reel's segments here
1290     (`_resolve_compile`), stages a
1291     small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
1292     to the bucket, dispatches a `compile` Cloud Run Job on the L4 (NVENC), bucket-
polls the
1293     done-marker, and returns the same `{status, filepath, download_url}` shape the SPA
expects. The
1294     worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
1295     `_maybe_dispatch_remote` (the DETECT dispatch)."""
1296     from backend.compute import CompileJobSpec, get_compute_backend
1297     notify("resolving")
1298     try:
1299         video, kept, out_name = _resolve_compile(
1300             db, cfg, video_id, segment_ids, output_filename
1301         )
1302     except _CompileError as e:
1303         return {"status": "failed", "message": e.detail, "video_id": video_id}
1304
1305     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1306     storage_cfg = getattr(cfg, "storage", None)
1307     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1308     compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
1309     if compute is None:
1310         # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
1311         # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
1312         # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
1313         logger.warning(
1314             "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
1315             "stitching in-process on the control-plane instead.",
1316             out_root,
1317         )
1318         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1319         return _stitch_reel(
1320             cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
1321         )
1322
1323     # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
1324     # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
to the L4's
1325     # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
1326     stitch_dump = stitching.model_dump()
1327     stitch_dump["encoder"] = getattr(
1328         getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
1329     )
1330     spec_payload = {
1331         "video_id": video_id,
1332         "video_uri": video["filepath"],
1333         "output_name": out_name,
1334         "locale": locale,
1335         "time_pairs": [

```

```

1336         [float(s["start_time"]), float(s["end_time"])] for s in kept
1337     ],
1338     "stitching": stitch_dump,
1339 }
1340 token = uuid.uuid4().hex
1341 spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1342 try:
1343     notify("staging compile spec")
1344     with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
1345         local_spec = os.path.join(td, "spec.json")
1346         with open(local_spec, "w", encoding="utf-8") as f:
1347             json.dump(spec_payload, f)
1348         storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1349 except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
1350     logger.error(
1351         "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
1352     )
1353     return {
1354         "status": "failed",
1355         "message": f"cloud compile: could not stage the compile spec: {e}",
1356         "video_id": video_id,
1357     }
1358
1359     # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
1360     # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
1361     overrides = [f"storage.output_root={out_root}"]
1362     backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
1363     if backend_name:
1364         overrides.append(f"storage.backend={backend_name}")
1365     spec = CompileJobSpec(
1366         out_uri=out_root,
1367         spec_uri=spec_uri,
1368         video_id=video_id,
1369         output_name=out_name,
1370         config_overrides=tuple(overrides),
1371     )
1372
1373     notify("dispatching stitch (cloud_run)")
1374     logger.info(
1375         "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
1376         video_id,
1377         out_root,
1378         len(kept),

```

#### 24. assistant (2026-07-08T06:15:59.638Z)

I now have the full producer/consumer contract. Producer stages spec keys `{video\_id, video\_uri, output\_name, locale, time\_pairs, stitching}` and consumer reads exactly those. Producer emits argv `[compile, --out-uri, --spec-uri, --done-uri, --set]` and parser defines exactly those (plus defaulted --config/--scratch). They match today. The claim is a *\*test-coverage\** one: no test round-trips producer?consumer. Let me examine the tests thoroughly.

#### 25. user (2026-07-08T06:16:02.261Z)

```

1     """Compile/stitch ? L4 worker dispatch (#521).
2
3     Proves the control-plane can OFFLOAD the reel compile/stitch to the Cloud Run L4 worker
by config
4     (`deployment.phase_placement.compile=cloud_run_l4`), mirroring the DETECT dispatch ?
all GPU-free +

```

```

5      cloud-free via a FAKE ``CloudRunJobClient`` that simulates the compile worker (reads the
staged
6      compile-spec + writes the reel + done-marker into a local "bucket"). Cardinal invariant:
DEFAULT
7      (compile on the control-plane) still stitches in-process, byte-identically.
8      ""
9
10     import json
11     import os
12
13     import pytest
14
15     import backend.compute as compute_pkg
16     import backend.orchestration as orch
17     from backend.compute.cloud_run import CloudRunJobClient
18     from backend.config import AppConfig
19     from backend.infrastructure.database import Database
20     from backend.infrastructure.execution_policy import ExecutionPolicy
21
22     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
23
24
25     class _FakeCompileClient(CloudRunJobClient):
26         """Simulates the Cloud Run compile worker: on dispatch, READ the staged compile-spec
(proving the
27         control-plane resolved + staged it), then write the reel done-marker into the local
bucket exactly
28         where the real worker would, so the dispatcher's bucket-poll resolves."""
29
30         def __init__(self, *, returncode=0, video_id=None):
31             self.calls: list[list[str]] = []
32             self.spec: dict | None = None
33             self.returncode = returncode
34             self._video_id = video_id
35
36         def run_job(self, job_name, argv, *, region, project=None):
37             self.calls.append(list(argv))
38             spec_uri = argv[argv.index("--spec-uri") + 1]
39             done_uri = argv[argv.index("--done-uri") + 1]
40             with open(spec_uri, encoding="utf-8") as f:
41                 self.spec = json.load(f)
42             vid = self._video_id or self.spec["video_id"]
43             out_name = self.spec["output_name"]
44             status = "success" if self.returncode == 0 else "failed"
45             marker = {
46                 "video_id": vid,
47                 "returncode": self.returncode,
48                 "status": status,
49                 "download_uri": f"/api/highlights/{vid}/{out_name}",
50                 "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
51                 "message": "" if self.returncode == 0 else "stitch blew up",
52             }
53             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
54             with open(done_uri, "w", encoding="utf-8") as f:
55                 json.dump(marker, f)
56             return "exec-compile-1"
57
58         def cancel_execution(self, execution): # pragma: no cover - happy path never
cancels
59             raise AssertionError("cancel must never fire on the happy path")
60
61
62     def _inject_fake(monkeypatch, fake):
63         real = compute_pkg.get_compute_backend
64         monkeypatch.setattr(

```

```

65         compute_pkg,
66         "get_compute_backend",
67         lambda config, **kw: real(config, run_client=fake, policy=_FAST, token="tok",
68         **kw),
69     )
70
71     @pytest.fixture
72     def compile_env(tmp_path, monkeypatch):
73         """A control-plane DB with a video + 3 play segments, wired for cloud with compile
74         placed on the
75         L4 and a LOCAL bucket (so the fake worker's spec/marker are plain file
76         reads/writes)."""
77         db = Database(str(tmp_path / "t.db"))
78         bucket = tmp_path / "bucket"
79         bucket.mkdir()
80         video_id = "vidCompile"
81         src = tmp_path / "source.mp4"
82         src.write_bytes(b"SRC")
83         db.add_video(video_id, str(src), "processed", [])
84         sid1 = db.add_play_segment(video_id, 2.0, 6.0, metadata={"shot_count": 5})
85         sid2 = db.add_play_segment(video_id, 10.0, 14.0, metadata={"shot_count": 3})
86         sid3 = db.add_play_segment(video_id, 20.0, 22.0, metadata={"shot_count": 4})
87         cfg = AppConfig.model_validate(
88             {
89                 "deployment": {
90                     "compute_target": "cloud_run",
91                     "phase_placement": {"compile": "cloud_run_l4"},
92                 },
93                 "storage": {"backend": "local", "output_root": str(bucket)},
94             }
95         )
96         return db, cfg, video_id, [sid1, sid2, sid3], bucket, monkeypatch
97
98     def _job(video_id, segment_ids, name="reel.mp4", locale=None):
99         return {
100             "id": "cjl",
101             "video_id": video_id,
102             "kind": "compile",
103             "params": json.dumps(
104                 {"segment_ids": segment_ids, "output_filename": name, "locale": locale}
105             ),
106         }
107
108     def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
109         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
110         fake = _FakeCompileClient()
111         _inject_fake(monkeypatch, fake)
112
113         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "myreel.mp4"))
114
115         assert out["status"] == "success"
116         assert out["video_id"] == video_id
117         assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
118         assert out["compute"]["path"] == "cloud_run"
119         assert out["compute"]["execution"] == "exec-compile-1"
120         assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
121
122     def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
123         """The staged compile-spec carries the RESOLVED time-pairs (padding-agnostic
124         start/end from the DB)
125         and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so

```



```

the worker
126     hardware-encodes and needs no control-plane DB."""
127     db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
128     fake = _FakeCompileClient()
129     _inject_fake(monkeypatch, fake)
130
131     orch._execute_compile(cfg, db, _job(video_id, seg_ids))
132
133     assert fake.spec is not None
134     assert fake.spec["video_id"] == video_id
135     assert fake.spec["video_uri"].endswith("source.mp4")
136     # 3 resolved (start, end) pairs from the DB rows
137     assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
138     # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
139     assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
140     # the argv is a `compile` subcommand carrying the spec + done marker
141     argv = fake.calls[0]
142     assert argv[0] == "compile"
143     assert "--spec-uri" in argv and "--done-uri" in argv
144     joined = " ".join(argv)
145     assert "storage.output_root=" in joined # bucket forwarded so the worker mirrors
the reel
146
147
148     def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
149         """The worker Job has no DEPLOYMENT_PROFILE, so the dispatch forwards the bucket
coordinates
150         (storage.output_root/backend) ? else the worker can't read the spec or mirror the
reel."""
151         db, cfg, video_id, seg_ids, bucket, monkeypatch = compile_env
152         fake = _FakeCompileClient()
153         _inject_fake(monkeypatch, fake)
154         orch._execute_compile(cfg, db, _job(video_id, seg_ids))
155         joined = " ".join(fake.calls[0])
156         assert f"storage.output_root={bucket}" in joined
157         assert "storage.backend=local" in joined
158
159
160     def test_compile_cloud_failure_returns_failed(compile_env):
161         """A worker stitch failure (rc != 0) surfaces as a failed job with the worker's
message ? the SPA
162         gets a clean verdict, not a crash."""
163         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
164         fake = _FakeCompileClient(returncode=3)
165         _inject_fake(monkeypatch, fake)
166
167         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids))
168         assert out["status"] == "failed"
169         assert "stitch blew up" in out["message"]
170         assert out["video_id"] == video_id
171
172
173     def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
174         """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
_stitch_reel
175         and NEVER dispatches to Cloud Run. The stitch core is stubbed so no real ffmpeg
runs."""
176         db, cfg, video_id, seg_ids, _bucket, _mp = compile_env
177         # flip compile back to the control-plane (the default) ? everything else stays
cloud-wired
178         cfg.deployment.phase_placement.compile = "control_plane"
179         fake = _FakeCompileClient()
180         _inject_fake(monkeypatch, fake)
181         seen = {}
182

```

```

183     def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify):
184         seen["pairs"] = pairs
185         seen["source"] = source
186         return {"status": "success", "video_id": vid, "download_url":
f"/api/highlights/{vid}/{name}"}
187
188     monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
189
190     out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "local.mp4"))
191     assert out["status"] == "success"
192     assert fake.calls == [] # NEVER dispatched ? the control-plane stitched in-process
193     assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
194     assert seen["source"].endswith("source.mp4")
195
196
197     def test_compile_missing_video_fails_before_dispatch(compile_env):
198         """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
199         db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
200         fake = _FakeCompileClient()
201         _inject_fake(monkeypatch, fake)
202         out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
203         assert out["status"] == "failed"
204         assert fake.calls == [] # never dispatched ? resolve failed first
205

```

## 26. assistant (2026-07-08T06:16:41.780Z)

`test\_compile\_dispatch.py` is a strong integration test: it injects a `\_FakeCompileClient` that subclasses the real `CloudRunJobClient`, so the REAL `orchestration.\_dispatch\_compile\_remote` ? REAL `CloudRunCompute.run\_compile` runs, producing the real argv and real staged spec. The fake reads `--spec-uri`/`--done-uri` from argv and `video\_id`/`output\_name`/`video\_uri`/`time\_pairs`/`stitching.encoder` from the real spec ? so a producer-side key/flag drift WOULD break these tests. But the fake is a re-implementation of the worker, not the real `build\_parser`+`cmd\_compile`. Let me verify the worker-side and compute-backend tests, and confirm there is no shared-constants module.

## 27. user (2026-07-08T06:16:49.861Z)

```

=== test_worker.py compile refs ===
84:     args = wmain.build_parser().parse_args(
97:     assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
104:     assert wmain.build_parser().parse_args(base).skip_validation is False
106:     assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
253:     args = wmain.build_parser().parse_args(
813:     args = wmain.build_parser().parse_args(
949:# ----- compile subcommand (#521 ? L4
stitch)
952: def test_parser_compile_accepts_spec_and_done_uris():
953:     args = wmain.build_parser().parse_args(
955:         "compile",
956:         "--spec-uri",
957:         "gs://b/_compile/tok/spec.json",
961:         "gs://b/_compile/tok.done",
966:     assert args.cmd == "compile"
967:     assert args.spec_uri == "gs://b/_compile/tok/spec.json"
968:     assert args.done_uri == "gs://b/_compile/tok.done"
972: def _write_compile_spec(bucket, video_id, source, name, encoder="h264_nvenc"):
975:     "video_uri": str(source),
976:     "output_name": name,
978:     "time_pairs": [[2.0, 6.0], [10.0, 14.0]],
981:     spec_dir = os.path.join(str(bucket), "_compile", "tok")
983:     spec_uri = os.path.join(spec_dir, "spec.json")
984:     with open(spec_uri, "w", encoding="utf-8") as f:
986:         return spec_uri

```

```

989: def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990:     """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core (VideoCompiler
stubbed so
991:     no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
998:     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
999:     done = str(bucket / "_compile" / "tok.done")
1003:     def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name, locale,
notify):
1005:         seen["pairs"] = time_pairs
1019:     rc = wmain.cmd_compile(
1020:         wmain.build_parser().parse_args(
1022:             "compile",
1023:             "--spec-uri",
1024:             spec_uri,
1048: def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1057:     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1058:     done = str(bucket / "_compile" / "tok.done")
1067:     rc = wmain.cmd_compile(
1068:         wmain.build_parser().parse_args(
1070:             "compile",
1071:             "--spec-uri",
1072:             spec_uri,
1089: def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
1097:     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1104:     rc = wmain.cmd_compile(
1105:         wmain.build_parser().parse_args(
1107:             "compile",
1108:             "--spec-uri",
1109:             spec_uri,
1120:     assert not (bucket / "_compile" / "tok.done").exists()

=== test_compute_backend.py compile refs ===
40:     def run_job(self, job_name, argv, *, region, project=None):
42:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
44:         out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
45:         done_uri = argv[argv.index("--done-uri") + 1]
84:     def run_job(self, job_name, argv, *, region, project=None):
86:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
105:     def run_job(self, job_name, argv, *, region, project=None):
107:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
131:     def run_job(self, job_name, argv, *, region, project=None):
133:         {"job": job_name, "argv": list(argv), "region": region, "project": project}
236: def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
252:     argv = client.calls[0]["argv"]
253:     assert argv[0] == "infer"
254:     assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
255:     assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
260:     assert "--skip-validation" not in argv
262:     joined = " ".join(argv)
272: def test_dispatch_argv_carries_skip_validation_when_set(tmp_path):
288:     assert "--skip-validation" in client.calls[0]["argv"]
301:     done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
302:     done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
745:     wmain.build_parser().parse_args(
803:     wmain.build_parser().parse_args(
868:     wmain.build_parser().parse_args(
922:     wmain.build_parser().parse_args(
979:     wmain.build_parser().parse_args(
1105: # ----- compile dispatch (#521 ? L4
stitch)
1110:     bucket exactly where the real ``cmd_compile`` would, so ``run_compile``'s bucket-poll
resolves."""
1116:     def run_job(self, job_name, argv, *, region, project=None):
1117:         self.calls.append({"job": job_name, "argv": list(argv)})

```

```

1118:         done_uri = argv[argv.index("--done-uri") + 1]
1131:         return "exec-compile"
1137: def test_run_compile_dispatches_and_returns_marker(tmp_path):
1138:     """run_compile dispatches a `compile` subcommand (spec + done marker), bucket-polls,
and returns
1140:     from backend.compute.base import CompileJobSpec
1147:     result = backend.run_compile(
1148:         CompileJobSpec(
1150:             spec_uri=f"{out}/_compile/tok/spec.json",
1159:     argv = client.calls[0]["argv"]
1160:     assert argv[0] == "compile"
1161:     assert argv[argv.index("--spec-uri") + 1] == f"{out}/_compile/tok/spec.json"
1162:     assert "--done-uri" in argv
1163:     # the CP-forwarded storage override rides as --set (compile carries NO infer container
overrides)
1164:     assert "storage.output_root=" + out in " ".join(argv)
1165:     assert "deployment.compute_target=local_gpu" not in " ".join(argv)
1168: def test_run_compile_surfaces_nonzero_returncode(tmp_path):
1171:     from backend.compute.base import CompileJobSpec
1178:     result = backend.run_compile(
1179:         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1184: def test_compile_marker_prefix_distinct_from_infer(tmp_path):
1185:     """The compile marker lives under _compile/ (not infer's _dispatch/) so a compile + a
detection
1187:     from backend.compute.base import CompileJobSpec
1194:     backend.run_compile(
1195:         CompileJobSpec(out_uri=out, spec_uri=f"{out}/s.json", video_id="v",
output_name="r.mp4")
1197:     done_uri = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
1198:     assert "/"_compile/" in done_uri and "/"_dispatch/" not in done_uri
1201: def test_run_compile_default_backend_raises_not_implemented():
1203:     CloudRunCompute overrides run_compile) ? never a silent no-op."""
1204:     from backend.compute.base import CompileJobSpec, ComputeBackend, ComputeResult
1211:     _InProc().run_compile(
1212:         CompileJobSpec(out_uri="x", spec_uri="y", video_id="v", output_name="r.mp4")

```

## 28. assistant (2026-07-08T06:16:58.319Z)

Let me read the worker-side hand-built spec fixture and the parser test fully.

## 29. user (2026-07-08T06:17:01.371Z)

```

949     # ----- compile subcommand (#521 ?
L4 stitch)
950
951
952     def test_parser_compile_accepts_spec_and_done_uris():
953         args = wmain.build_parser().parse_args(
954             [
955                 "compile",
956                 "--spec-uri",
957                 "gs://b/_compile/tok/spec.json",
958                 "--out-uri",
959                 "gs://b",
960                 "--done-uri",
961                 "gs://b/_compile/tok.done",
962                 "--set",
963                 "storage.output_root=gs://b",
964             ]
965         )
966         assert args.cmd == "compile"
967         assert args.spec_uri == "gs://b/_compile/tok/spec.json"
968         assert args.done_uri == "gs://b/_compile/tok.done"
969         assert args.set == ["storage.output_root=gs://b"]

```

```

970
971
972 def _write_compile_spec(bucket, video_id, source, name, encoder="h264_nvenc"):
973     spec = {
974         "video_id": video_id,
975         "video_uri": str(source),
976         "output_name": name,
977         "locale": None,
978         "time_pairs": [[2.0, 6.0], [10.0, 14.0]],
979         "stitching": {"encoder": encoder, "watermark": {"enabled": False}},
980     }
981     spec_dir = os.path.join(str(bucket), "_compile", "tok")
982     os.makedirs(spec_dir, exist_ok=True)
983     spec_uri = os.path.join(spec_dir, "spec.json")
984     with open(spec_uri, "w", encoding="utf-8") as f:
985         json.dump(spec, f)
986     return spec_uri
987
988
989 def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990     """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core
991     (VideoCompiler stubbed so
992     no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
993     bucket-poll."""
994     import backend.orchestration as orch
995
996     bucket = tmp_path / "bucket"
997     bucket.mkdir()
998     source = tmp_path / "src.mp4"
999     source.write_bytes(b"SRC")
1000     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1001     done = str(bucket / "_compile" / "tok.done")
1002
1003     seen = {}
1004
1005     def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name, locale,
1006     notify):
1007         seen["encoder"] = stitching.encoder
1008         seen["pairs"] = time_pairs
1009         seen["source"] = source_ref
1010         notify("stitching")
1011         return {
1012             "status": "success",
1013             "video_id": video_id,
1014             "download_url": f"/api/highlights/{video_id}/{out_name}",
1015             "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
1016         }
1017
1018     monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1019
1020     cfg = tmp_path / "c.json"
1021     cfg.write_text('{"storage": {"backend": "local"}}')
1022     rc = wmain.cmd_compile(
1023         wmain.build_parser().parse_args(
1024             [
1025                 "compile",
1026                 "--spec-uri",
1027                 spec_uri,
1028                 "--out-uri",
1029                 str(bucket),
1030                 "--done-uri",
1031                 done,
1032                 "--config",
1033                 str(cfg),
1034                 "--scratch",

```

```

1032         str(tmp_path / "s"),
1033         "--set",
1034         f"storage.output_root={bucket}",
1035     ]
1036 )
1037 )
1038 assert rc == 0
1039 # the shared stitch core got the resolved pairs + the NVENC encoder from the spec
1040 assert seen["encoder"] == "h264_nvenc"
1041 assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0)]
1042 assert seen["source"] == str(source)
1043 marker = json.loads(open(done, encoding="utf-8").read())
1044 assert marker["returncode"] == 0 and marker["status"] == "success"
1045 assert marker["download_url"] == "/api/highlights/vidReel/reel.mp4"
1046
1047
1048 def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1049     """A stitch failure surfaces as rc 3 + a failed marker (rc != 0) so the dispatcher's
poll
1050     terminates and the SPA sees a clean failure."""
1051     import backend.orchestration as orch
1052
1053     bucket = tmp_path / "bucket"
1054     bucket.mkdir()
1055     source = tmp_path / "src.mp4"
1056     source.write_bytes(b"SRC")
1057     spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
1058     done = str(bucket / "_compile" / "tok.done")
1059
1060     monkeypatch.setattr(
1061         orch,
1062         "_stitch_reel",
1063         lambda *a, **k: {"status": "failed", "message": "ffmpeg blew up", "video_id":
"vidReel"},
1064     )
1065     cfg = tmp_path / "c.json"
1066     cfg.write_text('{"storage": {"backend": "local"}}')
1067     rc = wmain.cmd_compile(
1068         wmain.build_parser().parse_args(

```

### 30. assistant (2026-07-08T06:17:37.493Z)

I now have the complete picture. Let me verify the final structural facts before concluding.

The contract, verified byte-by-byte at `dac6736`:

```

**Producer argv** (`cloud_run.py:323-335`): `["compile", "--out-uri", "--spec-uri", "--done-
uri"]` + `["--set", kv]`.
**Consumer parser** (`worker_main.py:865-900`): `--spec-uri` (required), `--out-uri`, `--done-
uri`, `--config`, `--scratch`, `--set`. Every produced flag is accepted; the only required flag
(`--spec-uri`) is always emitted. No mismatch.

**Producer spec** (`orchestration.py:1330-1339`): keys `{video_id, video_uri, output_name,
locale, time_pairs, stitching}`.
**Consumer reads** (`worker_main.py:489-496`):
`spec.get("video_id"|"output_name"|"video_uri"|"locale"|"time_pairs"|"stitching")` ? all via
`.get()` with defaults. Every key matches. No mismatch.

```

The claim's structural observation is factually accurate ? there is no single test that pipes the *\*real\** producer output into `build\_parser()`+`cmd\_compile`. But this does not amount to a reportable defect, for several concrete reasons I confirmed in the code.

### 31. user (2026-07-08T06:17:49.739Z)

Structured output provided successfully